

Лабораторная работа №6

Отчёт

Коровкин Никита Михайлович

Содержание

1	Цель работы	5
2	Задание	6
3	Выполнение лабораторной работы	7
4	Ответ на контрольные вопросы	17
5	Выводы	18
	Список литературы	19

Список иллюстраций

3.1	Запускаем фоновые процессы	7
3.2	Смотрим список процессов	8
3.3	Избавляемся от заданий	9
3.4	процесс в другом терминале	9
3.5	смотрим все запущенные процессы	10
3.6	смотрим все запущенные процессы после завершения	10
3.7	смотрим идентификаторы процессов	11
3.8	смотрим все запущенные процессы после завершения	11
3.9	смотрим все запущенные процессы после завершения	12
3.10	смотрим все запущенные процессы после завершения	12
3.11	смотрим все запущенные процессы и завершаем	13
3.12	запускаем процесс ес	13
3.13	смотрим все запущенные процессы после завершения	14
3.14	смотрим все запущенные процессы после завершения	14
3.15	завершаем процессы разными способами	15
3.16	защита от сигнала	15
3.17	выравниваем приоритеты	16

Список таблиц

1 Цель работы

Получить навыки управления процессами операционной системы.

2 Задание

Выполнить два этапа лабораторной работы

3 Выполнение лабораторной работы

1. Работа с заданиями (jobs)

Сначала я получил права администратора, выполнив команду:

su - После этого я запустил несколько процессов. Сначала я запустил команду:

sleep 3600 & Эта команда “усыпляет” процесс на час, а символ & в конце означает, что процесс будет выполняться в фоновом режиме, не блокируя терминал.

Затем я выполнил:

dd if=/dev/zero of=/dev/null & Эта команда копирует бесконечный поток нулей (/dev/zero) в «чёрную дыру» (/dev/null). Таким образом, процесс нагружает процессор, но не создаёт файлов. Она также запущена в фоновом режиме.

После этого я запустил ещё одну команду:

sleep 7200(рис. ??)

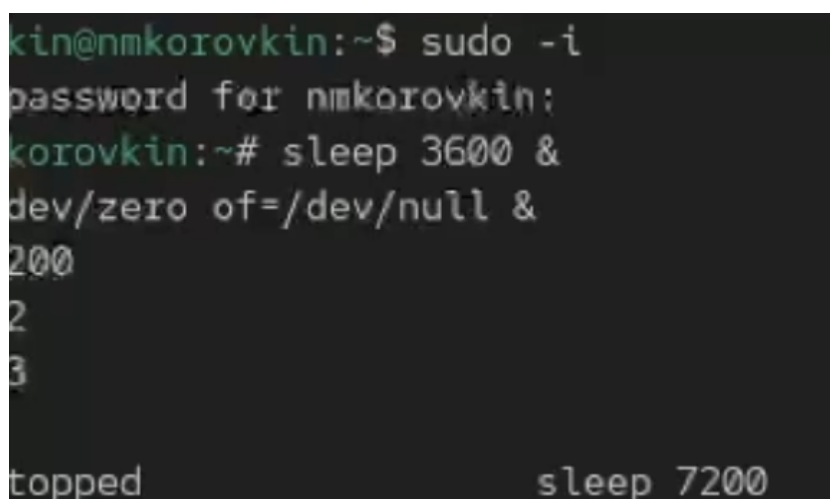
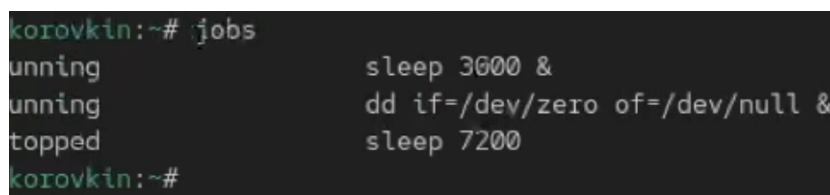
A screenshot of a terminal window with a dark background. The prompt is 'kin@nmkorovkin:~\$'. The user enters 'sudo -i', followed by the password 'nmkorovkin:'. The prompt changes to 'korovkin:~#'. The user enters 'sleep 3600 &', followed by 'dd if=/dev/zero of=/dev/null &'. The prompt returns to 'korovkin:~#'. The user enters 'sleep 7200'. The terminal shows the processes running in the background, with 'topped' and 'sleep 7200' visible at the bottom.

Рис. 3.1: Запускаем фоновые процессы

Эта команда усыпляет процесс на два часа, но без символа `&`, поэтому терминал «зависает» до окончания работы команды. Чтобы вернуть управление, я нажал `Ctrl + Z`, тем самым приостановив выполнение процесса.

Чтобы увидеть список всех запущенных мной заданий, я ввёл:

`jobs` В списке я увидел три задания: два в состоянии `Running` и одно в состоянии `Stopped` (приостановленное).(рис. ??)



```
korovkin:~# jobs
1 running      sleep 3600 &
2 running      dd if=/dev/zero of=/dev/null &
3 stopped      sleep 7200
korovkin:~#
```

Рис. 3.2: Смотрим список процессов

Чтобы возобновить приостановленное задание в фоновом режиме, я использовал: `bg 3` Теперь все процессы выполнялись в фоновом режиме. Проверив снова с помощью `jobs`, я увидел, что их состояние изменилось на `Running`.

Затем я переместил первое задание на передний план командой: `fg 1` После этого я прервал его выполнение с помощью `Ctrl + C`. Проверив список заданий, я убедился, что первое задание завершено.

Так же я последовательно завершил второе и третье задания. Для этого переводил их на передний план и останавливал с помощью `Ctrl + C`.(рис. ??)


```

korovkin:~# bg 3
sleep 7200 &
korovkin:~# jobs
running                sleep 3600 &
running                dd if=/dev/zero of=/dev/null &
running                sleep 7200 &
korovkin:~# fg 1
500
korovkin:~# jobs
running                dd if=/dev/zero of=/dev/null &
running                sleep 7200 &
korovkin:~# fg 2
dev/zero of=/dev/null
5835+0 records in
35+0 records out
39520 bytes (156 GB, 145 GiB) copied, 112.861 s, 1.4 GB/s
korovkin:~# fg 3
200
korovkin:~# jobs
korovkin:~#

```

Рис. 3.3: Избавляемся от заданий

2. Управление процессами в нескольких терминалах

Я открыл второй терминал и под своей учётной записью выполнил команду: `dd if=/dev/zero of=/dev/null &`. Эта команда снова запустила процесс в фоне.

После этого я закрыл второй терминал с помощью команды: `exit`. (рис. ??)

```

nmkorovkin@nmkorovkin:~$ dd if=/dev/zero of=/dev/null &
[1] 2961
nmkorovkin@nmkorovkin:~$ exit

```

Рис. 3.4: процесс в другом терминале

В другом(первом) терминале я запустил утилиту:

`top` Эта утилита показывает все запущенные в системе процессы в реальном времени. Я увидел, что процесс `dd` всё ещё работает, хотя терминал, из которого он был запущен, закрыт. Это объясняется тем, что процесс был запущен как фоновый и не был связан с управляющим терминалом напрямую.

Для выхода из top я нажал клавишу q.(рис. ??)

```
%Cpu(s):  6.4 us, 19.5 sy,  0.0 ni, 74.0 id,  0.0 wa,  0.0 hi,  0.0 si,  0.0 st
MiB Mem : 3899.9 total, 1285.0 free, 1399.1 used, 1406.1 buff/cache
MiB Swap: 2560.0 total, 2560.0 free,  0.0 used, 2500.8 avail Mem
```

PID	USER	PR	NI	VIRT	RES	SHR	S	%CPU	%MEM	TIME+	COMMAND
2961	nmkorov+	20	0	226660	1588	1588	R	100.0	0.0	0:32.80	dd
2644	nmkorov+	20	0	2691868	230588	89568	S	1.3	5.8	0:08.21	ptyxis
1846	nmkorov+	20	0	4946232	342916	115848	S	1.0	8.6	0:15.16	gnome-she+
2965	root	20	0	231848	4932	2884	R	0.7	0.1	0:00.08	top
1017	root	20	0	490504	26368	13184	S	0.3	0.7	0:00.41	tuned-ppd
2126	nmkorov+	20	0	615648	8412	8132	S	0.3	0.2	0:00.18	gpg-ident+

Рис. 3.5: смотрим все запущенные процессы

Затем я снова открыл top и с помощью клавиши k указал PID процесса dd, чтобы завершить его. После этого процесс исчез из списка.(рис. ??)

PID	USER	PR	NI	VIRT	RES	SHR	S	%CPU	%MEM	TIME+	COMMAND
2961	nmkorov+	20	0	226660	1588	1588	R	99.9	0.0	1:11.67	dd
2644	nmkorov+	20	0	2691868	231868	89568	S	14.9	5.8	0:14.31	ptyxis
1846	nmkorov+	20	0	4946216	342916	115848	S	6.5	8.6	0:18.70	gnome-she+
1644	root	20	0	158992	4472	4080	S	0.4	0.1	0:00.59	spice-vda+
3049	root	20	0	231848	4928	2880	R	0.3	0.1	0:00.04	top
13	root	20	0	0	0	0	I	0.1	0.0	0:00.39	kworker/u+
46	root	39	19	0	0	0	S	0.1	0.0	0:00.92	khugepaged
467	root	20	0	0	0	0	I	0.1	0.0	0:00.26	kworker/3+

Рис. 3.6: смотрим все запущенные процессы после завершения

3. Управление приоритетами процессов

Я снова получил права администратора и запустил три одинаковых процесса: dd if=/dev/zero of=/dev/null &. Чтобы увидеть их PID (идентификаторы процессов), я выполнил: ps aux | grep dd Команда ps aux показывает все запущенные процессы, а grep dd отбирает только те, где встречается «dd».(рис. ??)

```
root@nmkorovkin:~# dd if=/dev/zero of=/dev/null &
dd if=/dev/zero of=/dev/null &
dd if=/dev/zero of=/dev/null &
[1] 3274
[2] 3275
[3] 3278
root@nmkorovkin:~# ps aux | grep dd
root      2  0.0  0.0      0  0 ?        S   13:42   0:00 [kthreadd]
root     79  0.0  0.0      0  0 ?        I<  13:42   0:00 [kworker/R-ipv6
 _addrconf]
root     842  0.0  0.0 81072 3408 ?        Ssl 13:42   0:00 /usr/bin/qemu-g
a --method=virtio-serial --path=/dev/virtio-ports/org.qemu.guest_agent.0 --allow-r
pcs=guest-sync-delimited,guest-sync,guest-ping,guest-get-time,guest-set-time,guest
-info,guest-shutdown,guest-fsfreeze-status,guest-fsfreeze-freeze,guest-fsfreeze-fr
eeze-list,guest-fsfreeze-thaw,guest-fstrim,guest-suspend-disk,guest-suspend-ram,gu
est-suspend-hybrid,guest-network-get-interfaces,guest-get-vcpus,guest-set-vcpus,gu
est-get-disks,guest-get-fsinfo,guest-set-user-password,guest-get-memory-blocks,gue
st-set-memory-blocks,guest-get-memory-block-info,guest-get-host-name,guest-get-use
rs,guest-get-timezone,guest-get-osinfo,guest-get-devices,guest-ssh-get-authorized-
keys,guest-ssh-add-authorized-keys,guest-ssh-remove-authorized-keys,guest-get-disk
stats,guest-get-cpustats,guest-network-get-route -F/etc/qemu-ga/fsfreeze-hook
nmkorov+ 2169  0.0  0.6 1040936 24492 ?        Ssl 13:42   0:00 /usr/libexec/ev
```

Рис. 3.7: смотрим идентификаторы процессов

Я выбрал один из PID и изменил его приоритет командой:

renice -n 5 Команда renice изменяет приоритет процесса. Чем больше число, тем ниже приоритет (т.е. процесс получает меньше процессорного времени).(рис. ??)

```
root@nmkorovkin:~# renice -n 5 3278
3278 (process ID) old priority 0, new priority 5
root@nmkorovkin:~# ps fax | grep -B5 dd
```

Рис. 3.8: смотрим все запущенные процессы после завершения

Затем я ввёл:

ps fax | grep -B5 dd Опция -B5 показывает 5 строк до найденного совпадения. Это позволяет увидеть дерево процессов и родительский процесс, из которого были запущены мои dd. Я нашёл PID корневой оболочки (родительского процесса) и завершил её: kill -9 После этого оболочка закрылась, а вместе с ней прекратили работу все процессы dd. Это наглядно показывает, что если завершить родительский процесс, то все его дочерние процессы также остановятся.(рис. ??)

```

root@nmkorovkin:~# kill -9 3278
root@nmkorovkin:~# dd if=/dev/zero of=/dev/null &
[4] 3358
[3] Killed dd if=/dev/zero of=/dev/null &
root@nmkorovkin:~# dd if=/dev/zero of=/dev/null &
[5] 3364
root@nmkorovkin:~# dd if=/dev/zero of=/dev/null &
[6] 3370

```

Рис. 3.9: смотрим все запущенные процессы после завершения

4. Самостоятельная работа

Задание 1

Я трижды запустил процесс dd в фоне: dd if=/dev/zero of=/dev/null &. После этого я изменил приоритет одного из них, повысив его с помощью: renice -n -5 . Отрицательное значение приоритета означает, что процесс получает больший приоритет и выполняется быстрее.(рис. ??)

```

root@nmkorovkin:~# renice -n -5 3370
3370 (process ID) old priority 0, new priority -5
root@nmkorovkin:~# renice -n -15 3370
3370 (process ID) old priority -5, new priority -15
root@nmkorovkin:~# jobs
[1] Running dd if=/dev/zero of=/dev/null &
[2] Running dd if=/dev/zero of=/dev/null &
[4] Running dd if=/dev/zero of=/dev/null &
[5]- Running dd if=/dev/zero of=/dev/null &
[6]+ Running dd if=/dev/zero of=/dev/null &

```

Рис. 3.10: смотрим все запущенные процессы после завершения

Затем я ещё раз изменил приоритет того же процесса на -15. Разница заключается в том, что чем меньше (отрицательнее) значение nice, тем выше приоритет процесса. То есть процесс с приоритетом -15 будет выполняться с большей скоростью, чем с -5.

После проверки я завершил все запущенные процессы dd.(рис. ??)

```

root@nmkorovkin:~# jobs
[4]  Running                  dd if=/dev/zero of=/dev/null &
[5]-  Running                  dd if=/dev/zero of=/dev/null &
[6]+  Running                  dd if=/dev/zero of=/dev/null &
root@nmkorovkin:~# fg 4
dd if=/dev/zero of=/dev/null
^C269805853+0 records in
269805852+0 records out
138140596224 bytes (138 GB, 129 GiB) copied, 274.627 s, 503 MB/s

```

Рис. 3.11: смотрим все запущенные процессы и завершаем

Задание 2

Я запустил программу `yes` в фоновом режиме с подавлением вывода:

`yes > /dev/null &` Затем я запустил `yes` на переднем плане также с подавлением вывода и приостановил её с помощью `Ctrl + Z`. После этого я возобновил её командой `bg` и затем завершил с помощью `kill`. (рис. ??)

```

root@nmkorovkin:~# yes > /dev/null &
[1] 3581
root@nmkorovkin:~# yes > /dev/null
^Z
[2]+  Stopped                  yes > /dev/null
root@nmkorovkin:~# yes > /dev/null
^C

```

Рис. 3.12: запускаем процесс `es`

После этого я запустил `yes` без подавления вывода и снова приостановил выполнение, затем возобновил и остановил. Чтобы убедиться в текущих состояниях процессов, я использовал `jobs` (рис. ??)

```

y
y
y
y
y
y
^C
root@nmkorovkin:~# jobs
[1]  Running          yes > /dev/null &
[2]-  Stopped         yes > /dev/null
[3]+  Stopped         yes

```

Рис. 3.13: смотрим все запущенные процессы после завершения

Затем я перевёл один из фоновых процессов на передний план (fg) и завершил его. Другой процесс я снова запустил в фоне. Проверив jobs, я увидел, что процесс действительно находится в состоянии Running.(рис. ??)

```

root@nmkorovkin:~# jobs
[1]  Running          yes > /dev/null &
[2]-  Stopped         yes > /dev/null
[3]+  Stopped         yes
root@nmkorovkin:~# %1
yes > /dev/null
q
^Z
[1]+  Stopped         yes > /dev/null
root@nmkorovkin:~# fg %1
yes > /dev/null
^Z
[1]+  Stopped         yes > /dev/null
root@nmkorovkin:~# bg %1
[1]+ yes > /dev/null &

```

Рис. 3.14: смотрим все запущенные процессы после завершения

Чтобы запустить процесс, который будет продолжать работу даже после выхода из терминала, я использовал: `nohup yes > /dev/null &` После закрытия терминала и открытия нового я убедился, что процесс продолжает выполняться. Это возможно благодаря `nohup`, который игнорирует сигнал завершения сессии (SIGHUP).

Я просмотрел все процессы с помощью `top`, затем запустил ещё несколько `yes` в фоне и завершил два из них: один — по PID, другой — по номеру задания.(рис. ??)

```
nmkorovkin@nmkorovkin:~$ yes > /dev/null &
yes > /dev/null &
yes > /dev/null &
[1] 3790
[2] 3791
[3] 3792
nmkorovkin@nmkorovkin:~$ kill 3790
[1] Terminated yes > /dev/null
nmkorovkin@nmkorovkin:~$ kill %2
[2]- Terminated yes > /dev/null
nmkorovkin@nmkorovkin:~$
```

Рис. 3.15: завершаем процессы разными способами

Также я послал сигнал `SIGHUP` обычному процессу и процессу, запущенному с `nohup`. Первый завершился, а второй — продолжил работу, что подтверждает защиту от сигнала.(рис. ??)

```
nmkorovkin@nmkorovkin:~$ kill -1 3625
bash: kill: (3625) - Operation not permitted
nmkorovkin@nmkorovkin:~$ kill -1 2961
```

Рис. 3.16: защита от сигнала

После этого я запустил несколько процессов `yes` и завершил их все одновременно с помощью:

`killall yes`(рис. ??)

```
nmkorovkin@nmkorovkin:~$ yes > /dev/null &
yes > /dev/null &
yes > /dev/null &
[4] 3857
[5] 3858
[6] 3859
nmkorovkin@nmkorovkin:~$ killall yes
bash: killall: command not found...
nmkorovkin@nmkorovkin:~$ killall yes
yes(3625): Operation not permitted
[3] Terminated yes > /dev/null
[4] Terminated yes > /dev/null
[5]- Terminated yes > /dev/null
[6]+ Terminated yes > /dev/null
nmkorovkin@nmkorovkin:~$
```

Затем я снова запустил один процесс `yes` и ещё один с использованием `nice`, чтобы задать ему более высокий приоритет: `renice -n -5 yes > /dev/null &` После сравнения я заметил, что процесс с меньшим значением `nice` имеет более высокий приоритет. Наконец, я выровнял приоритеты двух процессов, используя: `renice -n 0` Теперь оба процесса имели одинаковые приоритеты.(рис. ??)

```
nmkorovkin:~# yes > /dev/null &
2797
nmkorovkin:~# nice -n 5 yes > /dev/null &
2818
nmkorovkin:~# ps aux | grep yes
2797 99.9 0.0 226624 1552 pts/1 R 14:30 0:47 yes
2818 100 0.0 226624 1552 pts/1 RN 14:30 0:38 yes
2841 0.0 0.0 227492 1876 pts/1 S+ 14:31 0:00 grep --color=auto yes
nmkorovkin:~# ps -o pid,ni,pri -p 2797,2818
  PID NI PRI
2797  0 19
2818  5 14
nmkorovkin:~# renice 0 -p 2818
process ID) old priority 5, new priority 0
nmkorovkin:~# ps -o pid,ni,pri -p 2797,2818
  PID NI PRI
2797  0 19
2818  0 19
nmkorovkin:~#
```

Рис. 3.17: выравниваем приоритеты

4 Ответ на контрольные вопросы

1. Команда: `jobs`

2. Комбинация: `Ctrl + Z`, затем команда

`bg`

3. Комбинация:

`Ctrl + C`

4. Использовать другую оболочку и выполнить:

`kill`

5. Команда: `ps fax`

6. Команда:

`renice -n -5 1234`

7. Команда: `killall dd`

8. Команда:

`killall mycommand`

9. В `top` — клавиша **k**

10. Команда:

`nice -n 10 <>`

5 Выводы

в результате выполнения работы мы научились работать процессами и управлять ими

Список литературы